TP 1 : MongoDB

idir.benouaret@epita.fr

1 Introduction et Contexte

MongoDB est une base de données NoSQL orientée documents qui se distingue par sa flexibilité et sa capacité à gérer de grandes quantités de données de manière efficace. Dans le cadre de ce TP, nous utiliserons MongoDB qui est la solution de stockage principale que vous utiliserez pour la réalisation du projet mini-Twitter. Grâce à son modèle de données basé sur BSON (une représentation binaire de JSON), MongoDB nous permet de stocker et de manipuler facilement des tweets sous forme de documents, sans être contraints par un schéma rigide. Cela facilite le stockage en masse des tweets, la gestion des timelines et l'optimisation des requêtes pour afficher rapidement le contenu pertinent aux utilisateurs. Son évolutivité et ses performances en lecture/écriture en font un choix idéal pour une application du type où les données sont fortement interconnectées et en constante évolution.

Ce TP a donc pour objectif de vous familiariser avec MongoDB et son shell. Vous allez pendant cette séance charger un jeu de données et écrire les requêtes permettant chacune de répondre à un besoin en information précis.

2 Installation et Préparation

Suivre les étapes suivantes pour installer et importer les jeux de données.
Remarque : Si vous êtes en salle machine de l'école : sauter l'étape 1 ci- dessous et utiliser la commande suivante pour lancer MongoDB

```
mkdir ~/mongodata
mongod -dbpath ~/mongodata
```

1. Télécharger, installer et lancer un serveur MongoDB :
   `https://www.mongodb.com/docs/manual/administration/install-community/`
   - Choisir l'installation correspondante à votre système d'exploitation.

- Par exemple pour Ubuntu :
  `https://www.mongodb.com/docs/manual/installation/#mongodb-community-edition`

2. Télécharger les fichiers suivants sur l'intra :
   - `tp1/movielens_movies.json`
   - `tp1/movielens_users.json`

3. Importer ces deux fichiers dans MongoDB avec les deux commandes illustrées ci dessous :

```
mongoimport --db MovieLens --collection movies --file movielens_movies.json
```

```
mongoimport --db MovieLens --collection users --file movielens_users.json
```

   Ces deux commandes permettent de créer les collections *movies* et *users* dans la base de données *MovieLens* et de les peupler avec les données issues des fichiers json.

4. Connectez-vous à la base de données MovieLens avec la commande *mongosh MovieLens*, qui lance le shell mongosh et se connecte à la base de données MovieLens. Ceci est illustré dans la figure 1

FIGURE 1: Lancement du shell mongo et connexion à la base MovieLens

5. Le shell *mongosh* est l'interface en ligne de commande interactive fournie par MongoDB pour interagir avec les bases de données MongoDB. Le shell propose des commandes spécifiques courantes dans les interfaces

6. **Schéma des données :**
   Les documents d'une collection ne sont pas soumis à un schéma fixe. Cependant, les documents de chaque collection ont une structure similaire. Le collection *movies* contient des données sur les films, c'est à dire un **identifiant** (id), un **titre** et un **genre**.

   La collection *users* contient des données sur les utilisateurs et les avis qu'ils ont donnés aux films qu'ils ont regardés. Parmi les données sur les utilisateurs, on trouve un **identifiant** (id), **nom**, **âge**, **occupation** et **sexe**. les notes données par chaque utilisateur sont représentées dans un **tableau de documents**, chacun de ces documents contient l'identifiant du film noté, la note attribuée par l'utilisateur ainsi que la date à laquelle l'utilisateur à exprimé sa note.

3 Exercices

Avant de commencer à répondre aux exercices, il faut attentivement lire les consignes suivantes.

Consignes

- Répondre à chacune des questions dans un fichier JavaScript avec l'extension *.js*
- Bien évidement c'est la requête qu'il faut soumettre et non pas son résultat.
- Le fichier *.js* doit contenir uniquement l'instruction mongosh qui permet de répondre à la question
- Chaque fichier doit être nommé de la façon suivante $exoNqM.js$, N étant le numéro d'exercice et M le numéro de la question.
- Par exemple la réponse à la question 3 de l'exercice 2 se fait dans le fichier $exo2q3.js$
- Pour toute requête `find` et `aggregate` renvoyant plusieurs documents, ajouter `.toArray()` à la fin de votre requête pour obtenir les résultats en tableau et non pas en curseur.
- Toutes les requêtes doivent être écrites en langage de requêtes MongoDB. **Les réponses qui récupèrent tous les documents et qui utilisent dessus un traitement en javascript ne seront pas acceptées**.

Exercice I : **Requêtes simples**

Q1 – Combien y a-t-il d'utilisateurs dans la base de données ?

Q2 – Combien y a-t-il de films dans la base de données ?

- `www.mongodb.com/docs/manual/reference/method/db.collection.countDocuments/`

Q3 – Quelle est l'occupation de l'utilisateur *Sherwood Kenneth* ? Écrire une requête dont la réponse affiche uniquement le nom et l'occupation.

- `www.mongodb.com/docs/manual/reference/method/db.collection.find/`

votre sortie doit avoir la forme suivante :

```
[ { name: 'Sherwood Kenneth', occupation: 'XXXX' } ]
```

Q4 – Combien d'utilisateurs ont entre 18 et 32 ans (inclus) ?

Q5 – Combien d'utilisateurs sont artistes (*artist*) ou écrivains (*writer*) ?

- `https://mongodb.com/docs/manual/reference/operator/query/or/`

Q6 – Quelles sont les dix femmes développeuses (occupation : programmer) les plus âgées ?

votre sortie doit contenir 10 documents et avoir la forme suivante :

```
[
{
name: '' ,
gender: '',
age: ,
occupation: ''
},
.
.
.

{
name: '',
gender: '',
age: ,
occupation: ''
}
]
```

Q7 – Quelles sont toutes les occupations présentes dans la bases de données. Votre requête doit renvoyer un **tableau** contenant toutes les occupations.

Exercice II : **Expressions régulières**

Q1 – Combien de films sont sortis dans les années 80 ? À noter que l'année de sortie est donnée entre parenthèse à la fin du titre de chaque film.

- ◇ `https://www.mongodb.com/docs/manual/reference/operator/query/regex/`

Q2 – Combien de films sont sortis entre 1982 et 1992 ?

Q3 – Combien y a-t-il de films *Thriller* (uniquement Thriller sans aucun autre genre) ?

Q4 – Trouver les films qui ont pour genre à la fois *"Action"* et *"Comedy"* ?

```
[
{
_id: ,
title: '',
genres: 'Action|Comedy|Drama'
},
.
.
.
{
_id: ,
title: '',
genres: 'Action|Comedy|Crime|Horror|Thriller'
}
]
```

Exercice III : **Insertion, mises-à-jour et suppressions**

Q1 – Insérer l'utilisateur suivant dans la base de données :

- ◇ name : "Aurelien Barrau"
- ◇ gender : "M"
- ◇ age : 50,
- ◇ occupation : "academic/educator"

Ne pas inclure pour l'instant le champ *movies*.

Q2 – Mettre à jour l'entrée insérée précédemment en ajoutant le champ *movies*, respectant le schéma adopté par les autres entrées. Ajouter les films dont les identifiants (*movieid*) sont 2628, 1210, 260, 1196 avec les notes (*rating*) 5, 4, 5 et 4 respectivement. Pour le champ *timestamp*, insérer 1705269118 pour les 4 entrées.

- ◇ `https://www.mongodb.com/docs/manual/tutorial/update-documents/`

Q3 – Supprimer l'entrée insérée précédemment de la collection

Q4 – Pour tous les utilisateurs qui ont pour occupation "*programmer*", changer cette occupation en *"developer"*.

- ◇ `https://www.mongodb.com/docs/manual/reference/method/db.collection.updateMany/`

Q5 – Le stockage de l'année de sortie dans le titre n'est pas très pratique. Modifier la collection *movies* en créant un champ *year* et en supprimant cette information du titre.

Par exemple, le document suivant

```
{ _id : 1721, title : "Titanic (1997)", genres : "Drama|Romance" }
```

doit être mis à jour de la façon suivante :

```
{ _id : 1721, title : "Titanic", genres : "Drama|Romance", year : 1997 }
```

Q6 – Modifier la collection *movies* en replaçant pour chaque film la valeur du champ genres par un tableau de chaînes de caractères

◇ `https://www.mongodb.com/docs/v4.2/reference/operator/aggregation/split/`

Sur l'exemple précédent cela donne :

```
{ _id : 1721, title : "Titanic", genres : ["Drama", "Romance"], year : 1997 }
```

Exercice IV : **Requêtes sur des tableaux**

Q1 – Combien d'utilisateurs ont noté le film *Star Wars : Episode V-The Empire Strikes Back* qui a pour id 1196 ?

◇ `https://www.mongodb.com/docs/manual/tutorial/query-array-of-documents/`

Q2 – Combien d'utilisateurs ont noté tous les films de la première trilogie Star Wars (id : 260, 1196, 1210) ?

◇ `https://www.mongodb.com/docs/manual/reference/operator/query/all/`

Q3 – Combien d'utilisateurs ont notés exactement 42 films ?

◇ `https://www.mongodb.com/docs/manual/reference/operator/query/size/`

Q4 – Pour chaque utilisateur, créer un champ *total_ratings* qui indique le nombre de films qu'il a notés.

Votre requête doit indiquer la mise à jour de tous les documents :

```
{
acknowledged: true,
insertedId: null,
matchedCount: 6040,
modifiedCount: 6040,
upsertedCount: 0
}
```

Q5 – Combien d'utilisateurs ont donné au film "Taxi Driver" la note de 5 ?

- ◇ https://www.mongodb.com/docs/manual/reference/operator/query/elemMatch/

Exercice V : **Agrégation**

Q1 – Montrer combien de films ont été produits durant chaque année des années 90 ; ordonner les résultats de l'année la plus fructueuse à la moins fructueuse.

- ◇ https://www.mongodb.com/docs/manual/core/aggregation-pipeline/
- ◇ https://www.mongodb.com/docs/manual/reference/operator/aggregation-pipeline/

Votre sortie doit avoir la forme suivante, avec $AAAA$ étant une année par exemple 1994 et X un entier :

```
[
{ _id: AAAA, count: X },
{ _id: AAAA, count: X },
{ _id: AAAA, count: X },
{ _id: AAAA, count: X },
{ _id: AAAA, count: X },
{ _id: AAAA, count: X },
{ _id: AAAA, count: X },
{ _id: AAAA, count: X },
{ _id: AAAA, count: X },
{ _id: AAAA, count: X }
]
```

Q2 – Quelle est la note moyenne du film Pulp Fiction, qui a pour id 296 ?

Vous ne devez pas arrondir le résultat, retournez simplement la moyenne telle que calculée

- ◇ https://www.mongodb.com/docs/manual/reference/operator/aggregation/unwind/

Votre sortie doit avoir la forme suivante :

```
[ { _id: 296, moyenne: x.xxx } ]
```

Q3 – Retourner pour chaque utilisateur son id, son nom, les notes maximale, minimale et moyenne qu'il a données, et ordonner le résultat par note moyenne croissante

Le résultat de votre requête doit être de la forme suivante :

```
[
{
maxRating: ,
minRating: ,
avgRating: ,
userId: ,
userName: ''
},
.
.
{
maxRating: ,
minRating: ,
avgRating: ,
userId: ,
userName: ''
}]
```